BuildSphere AI

ENTERPRISE CONSTRUCTION PORTAL

Local Setup & Database Migration Guide

Target: Local Development Environment

1. Quickstart: Running Manually on Another Machine

Follow these linear steps to launch both the Django backend and Vite React frontend.

Step 1: Environment & Virtualenv

```
cd Final_Project
python3 -m venv env
source env/bin/activate # On Windows: env\Scripts\activate
pip install -r requirements.txt
```

Step 2: Choose Database in .env

```
cp .env.example .env
# To use SQLite (zero setup): in .env set USE_SQLITE=True
# To use PostgreSQL: keep USE_SQLITE=False & run docker compose up -d db
```

Step 3: Run Database Migrations

```
cd backend
python manage.py migrate
```

Step 4: Create Initial Admin Account

```
python manage.py create_admin --email admin@buildsphere.com --password ChangeMe#12345
```

Step 5: Start Django Backend (Port 8000)

```
python manage.py runserver 0.0.0.0:8000
# API Swagger UI will be ready at http://localhost:8000/api/docs/
```

Step 6: Start Vite Frontend (Port 3000)

```
cd ../frontend
npm install
npm run dev
# Web app opens at http://localhost:3000 (Sign in: admin@buildsphere.com / ChangeMe#12345)
```

2. Moving Database from PostgreSQL to SQLite

BuildSphere AI includes a built-in switch supporting SQLite without code changes.

Option A: Clean SQLite Start (Zero Dependencies / No Postgres Needed)

1. In **.env** set: USE_SQLITE=True
2. Run migrations: python backend/manage.py migrate (generates backend/db.sqlite3)
3. Create admin: python backend/manage.py create_admin --email admin@buildsphere.com --password ChangeMe#12345
4. Start server: python backend/manage.py runserver 8000

Option B: Transfer Existing PostgreSQL Data into SQLite

1. Export from PostgreSQL (while connected to Postgres):
python manage.py dumpdata --natural-foreign --natural-primary -e contenttypes -e auth.Permission --indent 2 > datadump.json

2. Switch Configuration to SQLite:
In **.env** change: USE_SQLITE=True

3. Initialize SQLite Schema:
rm -f db.sqlite3
python manage.py migrate

4. Import Exported Data into SQLite:
python manage.py loaddata datadump.json
rm datadump.json